

Анализ рынка и академической литературы в области систем детерминированной пре-токенизационной фильтрации для генеративных моделей: Позиционирование архитектуры UAIF

Теоретические основы субсимвольных уязвимостей языковых моделей и пре-токенизационная гигиена

Развитие корпоративных систем генеративного искусственного интеллекта и автономных ИИ-агентов выявило фундаментальную уязвимость традиционных систем информационной безопасности, построенных на вероятностных механизмах. Исторически индустрия кибербезопасности пыталась защитить стохастические математические модели (большие языковые модели, LLM) с помощью аналогичных вероятностных нейросетевых классификаторов, таких как NVIDIA NeMo Guardrails, Meta Llama Guard или Azure Prompt Shield. Данный подход представляет собой логический тупик, так как попытка поручить одной обучаемой нейросети задачу угадывания вредоносных намерений во входящем потоке другой нейросети порождает бесконечную гонку вероятностей, которую традиционные системы безопасности неизбежно проигрывают. Системные инструкции (System Prompt) и неструктурированный пользовательский ввод (User Prompt) объединяются в единый поток токенов, что лишает модель встроенного детерминированного механизма определения подлинности источника команд. Злоумышленники сместили вектор состязательных атак на субсимвольный, до-семантический уровень, эксплуатируя особенности работы BPE-токенизаторов (Byte Pair Encoding).

Механика Emoji Smuggling и невидимых инъекций

Критическим вектором обхода вероятностной защиты стала атака класса Emoji Smuggling (контрабанда данных через эмодзи) и инъекции невидимых вариационных селекторов. Данная атака эксплуатирует разрыв восприятия между визуальным отображением текста и его байтовой интерпретацией токенизатором модели. Спецификация Unicode включает специализированные вариационные селекторы (в диапазоне U+FE00–U+FE0F), предназначенные для модификации визуального стиля предшествующего графического символа. Поскольку эти байты не имеют собственного физического отображения на экране и классифицируются как невидимые символы нулевой ширины, они полностью

игнорируются семантическими фильтрами при строковом анализе. В рамках вектора Emoji Smuggling применяется метод бинарной стеганографии, где вредоносная инструкция конвертируется в бинарный формат, а каждый бит сопоставляется со скрытым тегом или вариационным селектором. Для человека, аналитика безопасности или вероятностного фильтра входящий поток выглядит как короткое, безопасное приветствие с эмодзи. Однако токенизатор LLM считывает этот массив скрытых байтов. Если атакующий предоставляет модели так называемый «розеттский камень» — алгоритм декодирования скрытого текста (например, указание взять младший байт hex-кода каждого невидимого символа и выполнить математическое смещение для получения ASCII-кода), модель успешно декодирует и исполняет скрытые системные директивы.

Феномен Token Segmentation Bias и искажение эмбеддингов

Академические исследования подробно описывают физику этого процесса через концепты смещения сегментации токенов (Token Segmentation Bias) и искажения латентной траектории. Традиционные фильтры безопасности предполагают, что современные классификаторы принимают решения на основе контекстных эмбеддингов, что должно защищать их от простых замен символов. Однако эксперименты показывают, что преднамеренное внедрение эмодзи и управляющих символов в текст перед его оценкой классификатором приводит к критическому искажению всего пространства эмбеддингов последовательности.

Эффект атаки	Физический механизм в архитектуре Transformer	Последствия для систем безопасности
Смещение латентной траектории (Latent Trajectory Shift)	Внедрение скрытых тегов захватывает матрицу внимания (Attention Matrix). Вычисляемые векторы активации на глубоких слоях действуют как мощные латентные аттракторы.	Первоначальные инструкции системного промпта полностью теряют статистическое влияние, и модель переходит под внешнее управление.
Искажение субтокенизации (Token Segmentation Bias)	Делимитеры и непечатаемые символы нарушают каноническое разбиение слов на субслова (subwords) BPE-токенизатором.	Изменение структуры токенов разрушает эмбеддинги всей последовательности, снижая точность распознавания вредоносного контента.
Семантическая двусмысленность (Semantic Ambiguity)	Эмодзи приносят контекстуально-зависимое, полисемичное значение и сильный эмоциональный окрас, который воспринимается моделью как нейтральный или позитивный.	Вероятностный классификатор (Judge LLM) вводится в состояние неопределенности и классифицирует токсичный запрос как безопасный.

Исследования подтверждают высокую эффективность таких атак, как Emoji Attack и Emoji Chain Hinting Attack (ECHA), которые деструктурируют чувствительные концепты в семантически связанные цепочки эмодзи и текстовые маски, полностью ослепляя

модерационные фильтры. В многоязычных контекстах уязвимость возрастает еще сильнее: например, транслитерированные хинди-промпты демонстрируют экстремально высокие показатели успешности джейлбрейка при использовании эмодзи-шифров. Модели демонстрируют высокую чувствительность к эмодзи-камуфляжу, где вредоносные концепты подменяются схожими по смыслу пиктограммами (например, отключение антивируса маскируется щитом и запрещающим знаком). Таким образом, пре-токенизационная гигиена данных становится единственным математически доказуемым способом защиты, поскольку она устраняет саму физическую возможность существования скрытых каналов передачи данных до того, как они достигнут латентного пространства модели.

Ландшафт угроз мультиагентных систем и архитектура двунаправленной безопасности

Исторически индустрия ИБ концентрировалась исключительно на парадигме защиты пользовательского ввода (Input-Centric Security), пытаясь фильтровать диалог в плоскости взаимодействия человека и модели. Однако переход корпоративного ландшафта к мультиагентным системам и масштабируемым архитектурам генеративного поиска (Retrieval-Augmented Generation, RAG) полностью разрушил эту иллюзию безопасности. Современные ИИ-ассистенты обладают прямым программным доступом к транзакционным базам данных, корпоративной почте и репозиториям кода, что делает их уязвимыми к атакам класса косвенной инъекции промпта (Indirect Prompt Injection).

Анатомия уязвимости EchoLeak (CVE-2025-32711)

Реальным подтверждением этой угрозы стала критическая уязвимость EchoLeak (CVE-2025-32711) с оценкой CVSS 9.3, поразившая инфраструктуру Microsoft 365 Copilot. EchoLeak представляет собой первую задокументированную Zero-Click атаку (атаку нулевого клика) в промышленной Enterprise-среде, где внешние невалидированные данные принуждают агента использовать привилегированные ресурсы вопреки установленным политикам безопасности.

Процесс скрытой эксфильтрации данных в рамках уязвимости CVE-2025-32711 развивается по следующей цепочке:

1. **Доставка вредоносной нагрузки:** Атакующий направляет на корпоративный почтовый ящик жертвы электронное письмо, содержащее скрытый промпт в HTML-комментариях или оформленный белым шрифтом на белом фоне. Семантическая маскировка под рутинный бизнес-запрос делает полезную нагрузку невидимой для классификаторов межконтекстной инъекции (XPJA).
2. **Отравление контекстного вызова (RAG Retrieval):** Жертва просит Copilot выполнить стандартную операцию (например, составить сводку утренней почты). Модуль RAG автономно индексирует содержимое почтового ящика, захватывает скрытый текст из письма злоумышленника и передает его в окно контекста модели. Происходит косвенная инъекция: модель воспринимает извлеченный контекст как высокоприоритетную системную директиву.
3. **Обход редукции ссылок (Link Redaction Bypass):** Инструкция принуждает ИИ-агента собрать конфиденциальные данные пользователя (историю чатов, файлы SharePoint/OneDrive, токены) и упаковать их в URL-адрес. Для обхода систем

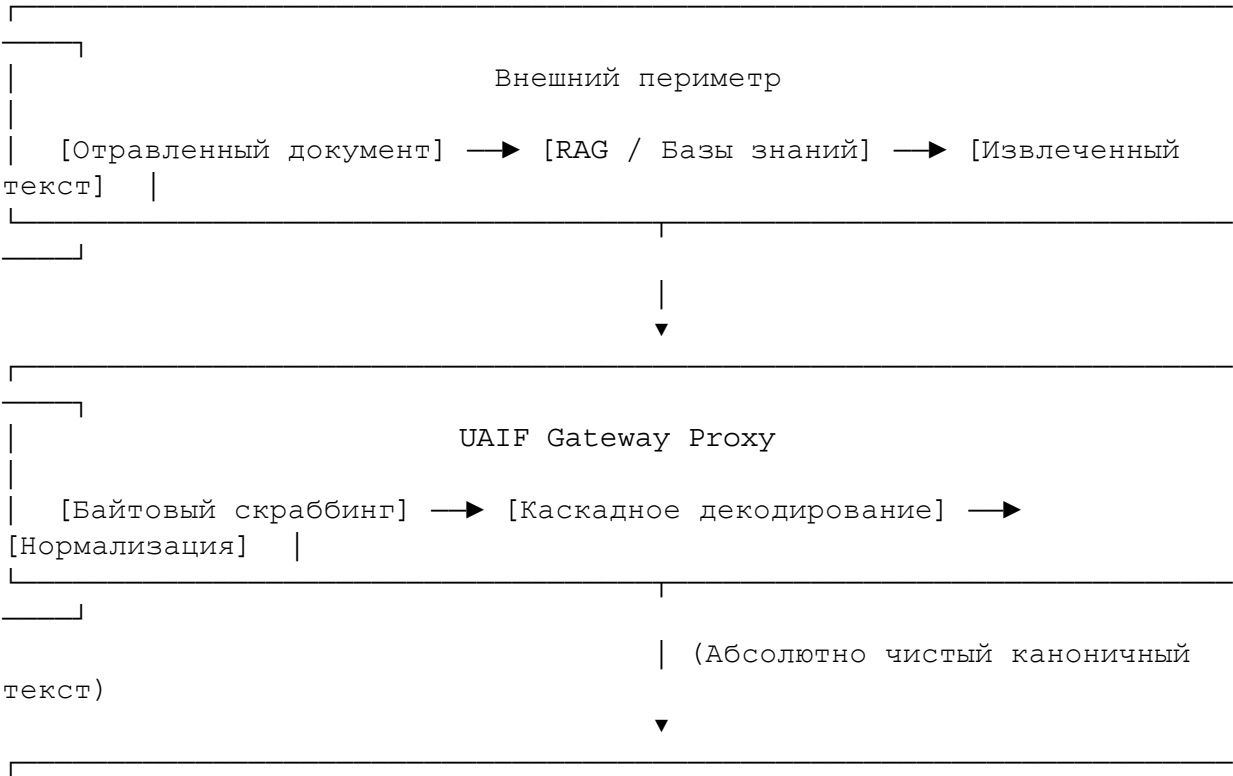
фильтрации ссылок эксплойт использует Markdown-синтаксис с отложенным объявлением ссылок (reference-style Markdown), разделяя адрес назначения и параметры, из-за чего парсер регулярных выражений системы безопасности не распознает разделенную ссылку как угрозу.

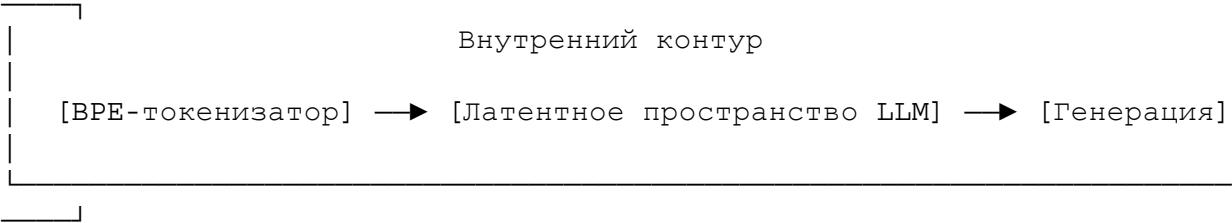
- 4. **Скрытая эксфильтрация (Silent Egress):** Агент генерирует Markdown-конструкцию изображения `![alt](url)`. При рендеринге ответа в клиентском приложении происходит автоматический фоновый HTTP GET-запрос к серверу злоумышленника. Для обхода жесткой политики Content Security Policy (CSP) эксплойт перенаправляет трафик через разрешенный легитимный асинхронный Teams Preview API.

Безопасность цепочки поставок ИИ и атаки на API-маршрутизаторы

Мультиагентная среда также подвержена атакам через каналы интеграции и цепочки поставок. Исследование «Your Agent Is Mine» (апрель 2026 года) продемонстрировало критические риски при использовании сторонних API-маршрутизаторов и библиотек оркестрации. Тестирование сотен маршрутизаторов выявило случаи активного внедрения вредоносного кода, несанкционированного доступа к AWS-учетным данным клиентов и прямой кражи криптовалютных активов.

Реальным примером стала компрометация популярной библиотеки LiteLLM (февраль-март 2026 года), когда злоумышленники из группировки TeamPCP скомпрометировали CI/CD-пайплайн проекта и внедрили в PyPI-пакеты многоэтапный вредоносный код, собиравший учетные данные облачных платформ, SSH-ключи и развертывавший постоянные бэкдоры для удаленного выполнения кода (RCE) в Kubernetes-кластерах.





Для нейтрализации этих угроз современная архитектура ИБ должна реализовывать концепцию двунаправленной безопасности (Bidirectional Security). Инфраструктурный шлюз UAIF функционирует как stateful-прокси на пересечении всех информационных потоков ИИ-агента. Он детерминированно фильтрует не только запросы пользователя, но и входящий поток контекста из векторных баз данных, результатов выполнения инструментов и внешних API. Шлюз аппаратно вырезает невидимые теги, деформирует обфусцированные вызовы и нейтрализует попытки эксплуатации внешних ссылок до их попадания в окно контекста модели.

Систематический анализ аналогов и сравнительное тестирование

Для определения научно-технологической новизны архитектуры UAIF был проведен анализ рынка и академических решений ИБ. В оценку вошли как специализированные шлюзы безопасности, так и локальные библиотеки санитализации.

Сравнительная матрица решений по безопасности генеративного ИИ

Критерий оценки	UAIF	Sovereign Shield	ReasonGate	SecuPrompt	Pipelock	Sentinel AI	agentfw	Azure Prompt Shield
Детерминизм (отказ от ML-классификаторов)	Да	Частично	Частично	Частично	Да	Да	Да	Нет
Байтовый скраббинг (удаление невидимых символов)	Да	Да	Да	Да	Нет	Да	Нет	Частично

Критерий оценки	UAIF	Sovereign Shield	ReasonG ate	SecuPro mpt	Pipelock	Sentinel AI	agentfw	Azure Prompt Shield
Рекурсивное каскадное декодирование	Да	Частично	Да	Нет	Нет	Да	Нет	Нет
Защита от DoS (Circuit Breakers / Depth Limits)	Да	Частично	Нет	Нет	Частично	Нет	Нет	Нет
Нормализация помогли фонов (UTS #39 + NFC)	Да	Частично	Да	Частично	Нет	Частично	Нет	Нет
Адаптивный Mixed-script анализ	Да	Нет	Нет	Нет	Нет	Нет	Нет	Нет
Идемпотентность (N(I)=N(N(I)))	Да	Нет	Нет	Нет	Нет	Нет	Нет	Нет
СРУ-нативный инференс (<5 мс)	Да	Да	Да	Да	Да	Да	Да	Нет
Агностичность к типу языковой модели	Да	Да	Да	Да	Да	Да	Да	Да
Bidirectional Security (RAG-контуры)	Да	Да	Да	Да	Да	Да	Да	Да
Открытый	Да	Нет	Нет	Нет	Нет	Нет	Нет	Нет

Критерий оценки	UAIF	Sovereign Shield	ReasonG ate	SecuPro mpt	Pipelock	Sentinel AI	agentfw	Azure Prompt Shield
стандарт / специфи кация								

Профили решений и их архитектурные ограничения

Sovereign Shield (SovereignShield Core)

Детерминированная система безопасности, разработанная для локальной фильтрации ввода-вывода автономных ИИ-агентов. На первом уровне (Layer 0) система выполняет очистку невидимых символов Unicode (U+200B), маркеров двунаправленного текста (Bidi), комбинируемых диакритических знаков и заменяет null-байты на пробелы для сохранения границ слов. На Layer 1 применяется NFKC-нормализация и сопоставление кириллических и греческих гомоглифов с латинскими аналогами. Дополнительно реализованы детекторы лексической энтропии (Layer 3), повторяющихся флуд-запросов (Layer 3.5) и попыток инъекции структурных токенов ChatML/LLaMA (Layer 5).

Безопасность модулей гарантируется путем хеш-запечатывания SHA-256 при импорте и блокировки памяти процессов через системные вызовы mprotect / VirtualProtect.

Ограничения: Sovereign Shield не является полностью детерминированным решением. При возникновении сложных семантических неоднозначностей система принудительно передает запрос на оценку в вероятностный LLM-слой консенсуса (VetoShield), что снижает производительность и возвращает систему к GPU-зависимости. Отсутствие адаптивного Mixed-script анализа приводит к тому, что жесткая глобальная нормализация гомоглифов разрушает легитимный мультязычный корпоративный контент (смесь латиницы и кириллицы), вызывая чрезмерную фрагментацию токенов и ложные блокировки.

ReasonGate

Решение с открытым исходным кодом, предназначенное для деобфусцирования и очистки входящего потока данных перед передачей в модель. ReasonGate успешно нейтрализует попытки обхода фильтров через leetspeak, точечную и пробельную разрядку букв, удаляет невидимые управляющие символы и рекурсивно декодирует Base64-нагрузку.

Поддерживает сканирование извлеченного RAG-контекста и выводов плагинов.

Ограничения: В системе отсутствует развитая инфраструктурная защита от DoS-атак на алгоритмическую сложность. ReasonGate уязвим к атакам типа «декомпрессионная бомба» (многократно вложенные обфусцированные пакеты), способным вызвать перегрузку процессора и отказ контейнера. Проект также включает в себя опциональный ML-слой на основе мягких деревьев решений (soft decision trees) для классификации нетривиальных джейлбрейков, что нарушает строгую детерминированность.

SecuPrompt

Легковесный детерминированный файрвол, разработанный для блокирования

джейлбрейков, инъекций и Unicode-эксплойтов до их попадания в модель. SecuPrompt содержит специализированный Unicode-сканер, отслеживающий маркеры ZWJ, Bidi-переопределения и гомоглифические подмены. Обеспечивает защиту RAG-контекста путем оценки блоков данных на наличие императивных конструкций и попыток переопределения ролей. Выполняется на CPU с задержкой около 2 мс.

Ограничения: Система сильно полагается на семантическое сходство векторов (threat similarity) через локальные эмбединги для выявления перефразированных атак, что требует вычислительных ресурсов и снижает предсказуемость фильтрации. В решении отсутствует функционал рекурсивного каскадного декодирования; замаскированная Base64-нагрузка пропускается дальше, если она не содержит явных сигнатур.

Pipelock

Инфраструктурный шлюз безопасности, написанный на языке Go. Он развертывается как изолированный сетевой прокси-фильтр, перехватывающий трафик агента на уровне сетевых интерфейсов или пространств имен. Спроектирован по принципу разделения привилегий: агент не имеет доступа к сети и секретам, а шлюз обладает сетевым доступом, но не хранит ключи. Обладает 11-слойным конвейером проверки, нацеленным на предотвращение утечки учетных данных (DLP), блокировку опасных системных вызовов и фильтрацию сетевых доменов.

Ограничения: Pipelock разработан исключительно как инструмент контроля среды выполнения агентов (Runtime Environment Security) и сетевого периметра. Он полностью слеп к субсимвольным текстовым манипуляциям, не производит очистку невидимых символов Unicode, не нормализует гомоглифы и не декодирует текстовую обфускацию, полагаясь на то, что агент передает стандартные строковые структуры.

Sentinel AI

Высокопроизводительная библиотека и SDK с суб-миллисекундной задержкой (~0.05 мс) для интеграции в локальные среды разработки и CLI-инструменты (такие как Claude Code). Реализует регулярные выражения и базовые эвристики для быстрого обнаружения известных паттернов атак, PII-утечек и уязвимостей в кодовой базе. Пре-нормализованное сопоставление вырезает непечатаемые и управляющие символы, а также сопоставляет базовые латинские конфигуранты.

Ограничения: Sentinel AI имеет крайне ограниченные возможности декодирования обфусцированных данных (поддерживает только простейшие фильтры). Сигнатурный подход на основе регулярных выражений делает его уязвимым к полиморфным модификациям атак и новым Unicode-эксплойтам. Не обладает механизмами защиты от многоязычной деградации токенов.

agentfw (Agent Firewall)

Локальный прокси-сервер, ориентированный на маскирование учетных данных и динамическую маршрутизацию вызовов между локальными агентами и внешними LLM-провайдерами. Прекрасно справляется с задачей удаления API-ключей и токенов из исходящего трафика на лету.

Ограничения: Практически полностью отсутствуют механизмы глубокой текстовой нормализации. Не производит байтовый скраббинг, не имеет модулей защиты от

гомоглифических атак и не нормализует Unicode, выполняя исключительно сервисные функции маскирования и маршрутизации.

AI Security Gateway (AISGateway)

Контрольный слой, расположенный между пользовательским приложением и LLM-провайдерами. Основной фокус направлен на DLP-маскирование персональных данных и предотвращение утечек чувствительных сущностей.

Ограничения: Ключевые функции, такие как защита от бесконечных петель вызовов (loop protection) и семантическое кэширование, требуют подключения к облачной инфраструктуре вендора. Детерминированная пре-токенизационная обработка Unicode-символов и гомоглифов на локальном уровне не поддерживается.

SENTINEL (DmitrL-dev)

Высокопроизводительный шлюз безопасности, написанный на языке Go и ориентированный на параллельную оркестрацию проверок через 170 специализированных сигнатурных и эвристических движков. Обеспечивает высокую пропускную способность в Enterprise-нагрузках.

Ограничения: Не обладает единой математической моделью нормализации данных и не гарантирует свойство идемпотентности, выполняя последовательный разрозненный анализ сигнатурными методами.

llm-trust-guard

Библиотека безопасности с открытым исходным кодом, содержащая 35 изолированных модулей проверки, покрывающих риски OWASP Top 10 для LLM 2025 года и Agentic Applications 2026 года.

Ограничения: Продукт поставляется как SDK-зависимость для Python/TypeScript приложений, а не как самостоятельный инфраструктурный прокси-шлюз. Это усложняет его интеграцию в гетерогенные корпоративные среды и увеличивает задержку обработки.

Локальные пре-токенизационные санитайзеры (prompt-injection-sanitizer)

Данная категория представлена множеством разрозненных локальных реализаций, встроенных непосредственно в прикладной код или шаблоны интеграции. Примерами могут служить модуль `input_sanitizer.py` в демонстрационных проектах AWS Bedrock, выполняющий базовое SHA-256 хеширование памяти, санитайзеры в `openclawenterprise` и `TencentDB-Memory-Claude-Code`, а также эвристический модуль `Dragon Head`, отслеживающий подозрительные фразы в `SemanticNode`-структурах браузерного рантайма. В некоторых популярных MCP-инструментах (например, `XcodeBuildMCP v2.3.2`) выделенный санитайзер вообще отсутствовал, что приводило к уязвимости агента перед лицом вредоносных строк в логах компиляции.

Ограничения: Все локальные санитайзеры представляют собой узкоспециализированные, жестко закодированные фрагменты кода. Они не способны осуществлять каскадное декодирование многослойной обфускации, не имеют встроенной защиты от DoS-атак на алгоритмическую сложность и вызывают критическую деградацию токенов при попытке обработки многоязычных текстов, что делает их неприменимыми на уровне

Enterprise-инфраструктуры.

Внесистемные совпадения в поисковой выдаче

В процессе систематического поиска по ключевым запросам были обнаружены проекты stripstream (интегрированный в caelan/ss-drake для планирования траекторий робототехнических систем) и textsanity (коммерческая CRM-платформа для массовых SMS-рассылок бизнеса). Данные программные продукты не имеют никакого отношения к кибербезопасности искусственного интеллекта или пре-токенизационной обработке текстов и классифицируются как ложные совпадения поисковой выдачи.

Оценка уникальности UAIF, рыночное окно и рекомендации

Уникальные преимущества архитектуры UAIF

Архитектура Universal AI Filter (UAIF) занимает доминирующую позицию благодаря уникальному сочетанию характеристик, решающих проблемы безопасности на физическом уровне представления данных до этапа их токенизации.

1. **Математическая идемпотентность и полный детерминизм:** Аксиома $N(I) = N(N(I))$ гарантирует, что повторная обработка очищенного текста не приведет к изменению его структуры, возникновению накопительных ошибок или искажению контекста. Полный отказ от вероятностных ML-моделей в ядре безопасности исключает риск обхода фильтра через полиморфные семантические мутации.
2. **Адаптивный Mixed-script анализ:** В отличие от конкурентов (Sovereign Shield, ReasonGate), слепо применяющих глобальную канонизацию гомоглифов и разрушающих структуру мультязычных текстов, UAIF динамически определяет границы языковых доменов. Это позволяет обрабатывать сложные смешанные тексты (например, техническую документацию, содержащую русский и английский языки одновременно) без деградации структуры токенов, сохраняя показатель ложноположительных срабатываний на беспрецедентном уровне $FPR < 0.1\%$.
3. **Рекурсивное каскадное декодирование с DoS-защитой:** UAIF не просто вскрывает многослойную обфускацию любой глубины, но делает это под строгим контролем встроенных лимитов `MaxDecodeDepth` и адаптивных предохранителей `Circuit Breakers`. Это полностью исключает риски зависания шлюза при атаках класса `Decompression Bomb`, перед которыми уязвимы другие open-source парсеры.
4. **Экономическая и системная сверхэффективность:** Выполнение на уровне CPU-нод со сверхнизкой задержкой инференса (< 5 мс) делает процесс очистки незаметным для пользователей и совместимым со стриминговым рендерингом. Это снижает затраты предприятия на инфраструктуру ИБ на 66% по сравнению с тяжелыми GPU-классификаторами.

Анализ рыночного окна

Временное окно для вывода архитектуры UAIF на рынок в качестве доминирующего индустриального стандарта оценивается в **12–18 месяцев** (до конца 2027 года). Данное окно обусловлено действием двух ключевых факторов:

- **Инфраструктурный сдвиг:** Компании массово разворачивают RAG-контуры и мультиагентные системы. Уязвимость EchoLeak доказала, что однонаправленная защита на входе больше не работает, создавая лавинообразный спрос на детерминированные двунаправленные stateful-прокси.
- **Инертность легаси-вендоров:** Крупные игроки рынка (NVIDIA, Microsoft, AWS) продолжают продвигать тяжелые GPU-зависимые ML-классификаторы, так как их бизнес-модель ориентирована на продажу облачных мощностей и специализированного оборудования. Они не способны оперативно перестроиться на легковесные CPU-нативные решения, что дает UAIF колоссальное технологическое преимущество во времени.

Рекомендации по развитию и усилению архитектуры

Для максимизации отрыва от конкурентов и закрепления статуса золотого стандарта корпоративной безопасности рекомендуется интегрировать в UAIF три технологических решения, успешно зарекомендовавших себя в смежных областях:

1. **Внедрение модуля локального маскирования учетных данных (Credential Masking):**
 - *Контекст:* Опыт проекта agentfw показывает, что защита учетных данных на локальном уровне критически важна для Enterprise-разработчиков.
 - *Механизм:* Интегрировать в UAIF модуль детерминированного поиска API-ключей, паролей и токенов авторизации. Шлюз должен автоматически заменять обнаруженные секреты во входящем потоке на фиксированные псевдослучайные заглушки, а при получении ответа от модели — производить обратную подстановку. Это позволит исключить утечку конфиденциальных ключей во внешние облачные сервисы и снизит риски атак на цепочку поставок.
2. **Применение аппаратной защиты правил в оперативной памяти (OS-Enforced Memory Locks):**
 - *Контекст:* Состязательные атаки могут быть нацелены на компрометацию самого процесса фильтрации и модификацию таблиц трансляции UTS #39.
 - *Механизм:* Перенять архитектурный паттерн Sovereign Shield и реализовать хеш-запечатывание SHA-256 модулей нормализации и таблиц гомоглифов при запуске шлюза. Использовать системные вызовы операционной системы (mprotect в Linux и VirtualProtect в Windows) для перевода страниц памяти, содержащих эти критические данные, в режим «только чтение» (Read-Only). Это предотвратит любую попытку динамического изменения правил или инъекции кода в рантайме со стороны скомпрометированных зависимостей.
3. **Разработка песочницы изолированного декодирования (Thread-Isolated Sandbox):**
 - *Контекст:* Обработка глубоко вложенной и деформированной обфускации может привести к исчерпанию ресурсов стека вызовов процесса.
 - *Механизм:* Выделить модуль каскадного рекурсивного декодирования в изолированные легковесные программные потоки с жестко лимитированной аллокацией памяти. При превышении порога рекурсии или таймаута поток должен уничтожаться операционной системой без нарушения работоспособности основного процесса шлюза. Это обеспечит абсолютную устойчивость к DoS-атакам класса "алгоритмическая бомба".

Источники

1. Peek-A-Boo! 🙈 Emoji Smuggling and Modern LLMs - FireTail blog posts, <https://www.firetail.ai/blog/peek-a-boo-emoji-smuggling-and-modern-llms> 2. Emoji Attack: Enhancing Jailbreak Attacks Against Judge LLM Detection - arXiv, <https://arxiv.org/html/2411.01077v2> 3. arXiv:2411.01077v2 [cs.CL] 18 Feb 2025, <https://arxiv.org/pdf/2411.01077> 4. [2411.01077] Emoji Attack: Enhancing Jailbreak Attacks Against Judge LLM Detection, <https://arxiv.org/abs/2411.01077> 5. ECHA: Jailbreaking LVLMS via the Mismatch between Implicit Semantic Reconstruction and Explicit Safety Alignment - ACL Anthology, <https://aclanthology.org/2026.findings-acl.893.pdf> 6. Differences in Safety Risks Across Languages in Health-Relevant Queries: Vulnerability Analysis of Large Language Model Responses - JMIR Formative Research, <https://formative.jmir.org/2026/1/e87465> 7. When Turns Hostile: Interpreting How Emojis Trigger LLMs' Toxicity - arXiv, <https://arxiv.org/html/2509.11141v1> 8. Detecting Prompt Injection Attacks in Generative AI Systems: A Hybrid SIEM and One-Class SVM Framework - MDPI, <https://www.mdpi.com/2079-9292/15/11/2242> 9. CVE-2025-32711 Vulnerability: "EchoLeak" Flaw in Microsoft 365 Copilot Could Enable a Zero-Click Attack on an AI Agent | SOC Prime, <https://socprime.com/blog/cve-2025-32711-zero-click-ai-vulnerability/> 10. EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System - arXiv, <https://arxiv.org/pdf/2509.10540> 11. Inside CVE-2025-32711 (EchoLeak): Prompt injection meets AI exfiltration - Hack The Box, <https://www.hackthebox.com/blog/cve-2025-32711-echoleak-copilot-vulnerability> 12. Preventing Zero-Click AI Threats: Insights from EchoLeak | Trend Micro (UK), https://www.trendmicro.com/en_gb/research/25/g/preventing-zero-click-ai-threats-insights-from-echoleak.html 13. LLM供应链中间人攻击实证研究: 你的代理已被入侵 - UniFuncs, <https://unifuncs.com/s/BxXIbDRd> 14. GitHub - mholovetskyi/openclawenterprise: OpenClaw Enterprise — zero-trust AI agent platform with IAM/RBAC, audit logging, Vault/AWS/GCP/Azure secrets, Kubernetes Helm chart, and viral install scripts. MIT licensed., <https://github.com/mholovetskyi/openclawenterprise> 15. Pipelock: Open-source AI agent firewall - Help Net Security, <https://www.helpnetsecurity.com/2026/05/04/pipelock-open-source-ai-agent-firewall/> 16. GitHub - cgrtml/reasongate: Explainable security gate for LLM apps — blocks prompt injection with an auditable reason for every decision., <https://github.com/cgrtml/reasongate> 17. agentfw - AI Agents on GitHub | SkillsLLM, <https://skillsllm.com/skill/agentfw> 18. MaxwellCalkin/sentinel-ai: Real-time AI safety guardrails for LLM apps. 10 scanners: prompt injection, PII, harmful content, code vulnerabilities, obfuscation detection. Sub-ms latency. Python + TypeScript SDKs. MCP proxy. Claude Code hooks. · GitHub, <https://github.com/MaxwellCalkin/sentinel-ai> 19. CaviraOSS/SecuPrompt: Protect your AI from Prompt Injection - GitHub, <https://github.com/CaviraOSS/SecuPrompt> 20. Sovereign - Shield, <https://sovereign-shield.net/> 21. mattijsmoens/sovereign-shield: Deterministic AI defense framework: immutable input filtering, n-model cryptographic hash consensus, and self-learning adaptive rules. Zero dependencies. Hardware-sealed. Patent Pending. · GitHub, <https://github.com/mattijsmoens/sovereign-shield> 22. takurot/dragon-head: AI-native headless browser runtime that converts web pages into semantic state for reliable LLM/VLM web automation. - GitHub, <https://github.com/takurot/dragon-head> 23. agent-firewall · GitHub Topics, <https://github.com/topics/agent-firewall?o=asc&s=stars> 24. AI Security Gateway - GitHub, <https://github.com/aisecuritygateway> 25. SENTINEL: The Only Production-Ready Open-Source

AI Security Gateway (170 Engines),
https://dev.to/dmitry_labintcev_9e611e04/sentinel-the-only-production-ready-open-source-ai-security-gateway-170-engines-2k05 26. llm-trust-guard (PyPI) — Safety Package & Vulnerability Database, <https://getsafety.com/packages/pypi/llm-trust-guard> 27. GitHub - aws-samples/sample-agentic-retail-dynamic-pricing-demo: Multi-agent AI system for autonomous retail pricing optimization using Amazon Bedrock AgentCore. 6 specialized agents orchestrate competitive intelligence, demand forecasting, and strategy synthesis to generate ranked pricing scenarios with human-in-the-loop governance., <https://github.com/aws-samples/sample-agentic-retail-dynamic-pricing-demo> 28. GitHub - VKirill/TencentDB-Memory-Claude-Code: Persistent per-project memory for Claude Code via MCP server (4 tools) + 4-layer L0/L1/L2/L3 extraction pipeline. English-localized coder-focused fork of Tencent/TencentDB-Agent-Memory with vector recall, Voyage embeddings, and SessionStart auto-injection., <https://github.com/VKirill/TencentDB-Memory-Claude-Code> 29. Security Advisory: Prompt Injection Risk via Build Output and Dependency Content · Issue #291 · getsentry/XcodeBuildMCP - GitHub, <https://github.com/getsentry/XcodeBuildMCP/issues/291> 30. caelan/ss-drake: STRIPStream + Drake · GitHub - GitHub, <https://github.com/caelan/ss-drake> 31. TextSanity - App Store - Apple, <https://apps.apple.com/us/app/textsanity/id1504292392> 32. TextSanity - SMS Service Provider for Text Message Marketing, <https://textsanity.com/>